Performance, Resource Utilization, and Cost Analysis of Web Applications: Virtual Machines vs. Docker Containers

SEN421 Cloud Architecture and Services — Fall 2026

Istanbul Aydın University | Department of Software Engineering

Topic 2, Part A: Hands-on Deployment

Emre Avcı

Student ID: B2105.090126

emreavci3@stu.aydin.edu.tr

Problem

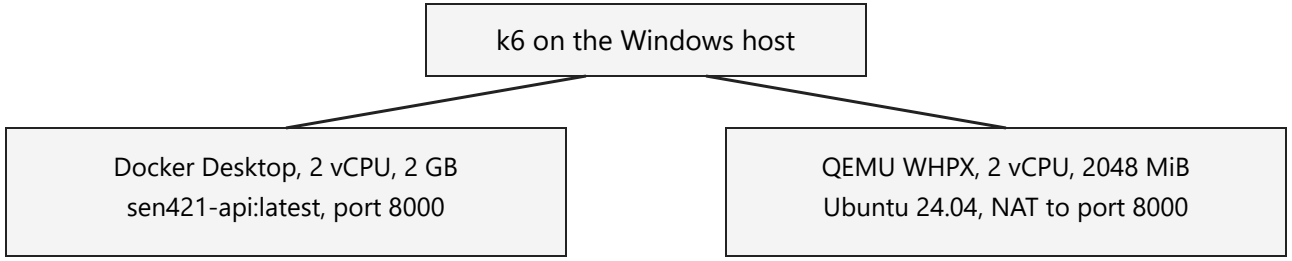
Same FastAPI service, two execution models.

Question: how much faster does a containerized API start and respond under the same load, and what are the trade-offs in resource overhead and isolation?

- Guest: private kernel, large disk, slow boot, stronger fault boundary
- Container: shared kernel, namespaces and cgroups, small image, fast start
- Cost is a projection. No AWS resource was created

Architecture

k6 calls `127.0.0.1:8000`. One target at a time. Same three routes, two uvicorn workers.



VirtualBox 7.2.22 fell back to NEM because VT-x was unavailable and did not finish boot.

Methodology

Property	Value
Load	k6 2.2.0, 50 VU, 30 s, `/compute` and `/payload`
Repetitions	3 runs, 15 s idle, sample standard deviation with n-1
Thresholds	HTTP 200 rate > 0.99 and p95 < 500 ms; failed runs were kept
Ready time	stop/start or process start until `/health` returns 200
Cost	AWS price list, eu-central-1, 730 h, accessed 21 October 2026
Traffic assumption	100,000 requests/day; outbound counts `/payload` only: 196.608 GB

Key Results: Latency and Throughput

Mean of three runs, plus or minus the sample standard deviation.

Metric	Docker	QEMU/WHPX
Median (ms)	224.121 ± 75.998	214.347 ± 70.009
p95 (ms)	585.389 ± 194.322	734.976 ± 67.310
Requests/s	174.925 ± 8.026	157.792 ± 6.847
Failed requests	not recorded in those JSON files	5.977% ± 1.149

The guest median is slightly lower. The spread is larger than the gap. The guest tail is slower and about 6% of its requests failed.

Key Results: Size, Boot, and Memory

Metric	Docker	QEMU/WHPX
Image or disk	inspect Size 49.701 MiB (column showed 208 MB)	file 2.792 GB; virtual size 64 GiB
Ready time (s)	1.450 ± 0.221	47.660 ± 6.244
Memory	idle 93.08 MiB; load peak 99.54 MiB	guest allocated 2048 MiB



Bar width is proportional to ready time. The guest disk file is about 56 times the image content.

Cost and Security

List price, 730 hours, free-tier allowance not subtracted. Actual experiment spend: 0 USD.

Item	EC2 t3.micro, 20 GB gp3	Fargate 0.25 vCPU, 0.5 GB
Compute	8.76000	10.36235
Storage	1.90400	0.00000
Data transfer out	17.69472	17.69472
Total (USD)	28.35872	28.05707

EC2: customer patches the guest. Fargate: AWS owns the host kernel; the customer owns the image. Frankfurt is outside Turkey (KVKK art. 9) if personal data were ever sent. This dataset is synthetic.

Lessons Learned

- Startup, size, and throughput favor the container. Median latency does not.
- The virtual machine's cost is mostly the second kernel, not the request.
- These runs are not a clean shared-kernel result: Docker Desktop uses a utility VM, and VirtualBox never left NEM.
- The priced Fargate shape is smaller than the measured 2 vCPU and 2 GB cap, and transfer dominates both bills.
- Next: one Ubuntu host with VT-x, VirtualBox, and Docker Engine, then Kubernetes for more than one replica.